百亿项斐波那契数的家用计算与十进制流式输出

陆旭东

（华东师范大学附属枫泾中学）

通信作者邮箱：lusunrise@126.com

在 32 GB 家用电脑上，复测第 100 亿项斐波那契数的计算与十进制完整输出。采用既有两平方递推、分治转换和独立进程诊断。两份不同哈希的 Windows GMP DLL 均在第 80 亿项最后一次平方失败。独立稀疏数平方和 C 复现器记录相邻结果：2,147,352,576 位成功，2,147,352,577 位出现 64 位分配尺寸回绕。WSL2 完成 3 次百亿项输出，Windows 原生 MPIR 完成 1 次。4 份输出与新生成的 Linux GMP 对照文件均含 2,089,876,403 位数字，SHA-256 一致。故障证据限于所测构建与输入，具体源码表达式尚未定位。

关键词：斐波那契数；任意精度整数；进制转换；流式输出；可复现计算

中图分类号：TP301.6　文献标志码：A

Computing and streaming the ten billionth Fibonacci number on a home computer

Xudong Lu

(Fengjing High School Affiliated to East China Normal University, Shanghai, China; lusunrise@126.com)

This study examines a reproducible workflow for computing and writing the ten billionth Fibonacci number on a 32 GB home computer. Established two-square recurrence and bounded-leaf decimal conversion are combined with isolated execution and explicit validation. New experiments used an Intel Core i5-14400F host, Windows 11 and WSL2 Ubuntu. Four arithmetic implementations were measured five times at each of four index sizes. Native Windows Python/GMP failed reproducibly at the same final square for n=8 billion in both gmpy2 2.2.1 and 2.3.1. WSL2 completed that index in three trials. At n=10 billion, three WSL2 trials had median arithmetic and conversion-with-close times of 56.95 s and 551.98 s; the median process peak RSS was 5.10 GiB. One native Windows MPIR v1.1.0 full-output run completed with process wall time 2592.93 s and observed peak working set 8.05 GiB. All four repair-path outputs and a newly generated native Linux GMP reference contain 2,089,876,403 digits and share one SHA-256 digest. Results concern the recorded builds and host, without establishing the exact internal cause of the original allocation failure.

Keywords: Fibonacci number; arbitrary precision integer; radix conversion; streaming output; reproducible computation

0 引言

斐波那契数的快速计算已有成熟数学方法和大整数库实现。对孤立的大下标数值，研究重点除了递推步骤，还包括整数运算的实际代价、内存分配和最终输出。任意精度算法中的乘法、除法和进制转换会采用与数据规模相关的实现策略[1]。GMP 的斐波那契函数使用按下标二进制位推进的算法，并公开给出每步两次平方的恒等式[2]；其大数乘法又会依据操作数规模和构建配置选择算法[3]。因此，递推层数少，并不意味着计算和输出可以忽略资源限制。

第 100 亿项斐波那契数具有约 21 亿位十进制数字。即使整数运算已经完成，一次性构造十进制字符串仍会增加约 2 GiB 的驻留对象，并叠加数值状态、转换临时量和库工作区。与此同时，应用界面和底层本地库处于同一进程时，底层分配失败可能直接使整个程序退出[4]。用户看到一个退出码，不能据此区分整数运算失败、转换失败和输出中断。

本文整理一项家用计算实验及其源代码[5]，研究算法分派、输出流程和诊断方法的共同作用。工作采用既有公式，贡献在于完整计算与文件输出的工程实现，以及可以检查的验证流程。本文不主张新递推算法，也不依据来源尚未认证的旧参考文件推断超算性能。

成果的公开价值在于完整、可检查的实验记录。普通 32 GB 家用机算出了 $F(10^{10})$，写出约 20.9 亿位十进制数字。两条路线的新输出与本次直接调用原生 GMP 生成的对照全文一致，旧参考文件另作历史核查。复现者可以检查源码、诊断记录、数字载荷和核验方法，而不仅是看到程序运行结束。

本项目从朴素递归和矩阵幂计算起步，先后尝试快速倍增、Lucas 及两平方算法。多次改进后，仍长期未能完成第 100 亿项的计算与输出。当时在 GitHub 作探索性检索，未找到可直接参照的同规模完整输出记录；检索日期和关键词未留存，因此不据此主张优先性。逐层日志把 Windows 失败定位到大整数平方；错误文本显示约 4 GiB，最初容易误认为

内存上限。后续全宽分配记录表明，实际请求为 2^{64}–369,632 字节。脱离递推的单次平方，以及直接调用同一 GMP DLL 的乘法，均复现这一异常。WSL2/Linux GMP 与 Windows 原生 MPIR 则完成了目标文件。GMP 开发记录曾报告旧版本超大整数乘法中的尺寸溢出[6]；本文不据此指定当前 DLL 的具体出错表达式。

1 递推方法与接口处理

1.1 两平方状态递推

令初始状态为第 1 项和第 0 项，并保存当前下标的奇偶性。处理目标下标除最高位之外的每个二进制位时，使用以下恒等式生成两倍下标附近的状态。

$$F(2k+1) = 4F(k)^2 - F(k-1)^2 + 2(-1)^k$$

$$F(2k-1) = F(k)^2 + F(k-1)^2$$

$$F(2k) = F(2k+1) - F(2k-1)$$

输入位为零时，保留偶数下标值及其前一项；输入位为一时，保留后一奇数下标值及其前一项。由此，当前状态下标始终等于已经读入的二进制前缀。第 100 亿项下标有 34 个二进制位，最高位用于初始化，循环执行 33 次。各层操作数不断增长，不能把 33 层理解为 33 次固定代价运算。

程序在每次平方完成后释放不再使用的数值引用，避免将整个递推历史保存在字典中。GMP 原生斐波那契函数的下标参数是无符号长整数[7]。本项目程序在下标不超过 4,294,967,294 时优先使用原生接口，超出后采用 Python 层两平方实现。该阈值是被检查程序采用的经验策略，不是对所有平台或绑定版本的统一上界。

1.2 算法选择的实验依据

本次比较原生 GMP、标准快速倍增、Lucas 和两平方。第 10 亿项的 5 次计时中位数分别为 3.844 s、7.844 s、5.622 s 和 4.610 s。计时协议见第 3 节。两平方在本机优于所测的另两种 Python 实现，原生接口更快；该结论限于记录的构建与规模。

表 1　整数运算 5 次测量的中位数

Table 1　Five-trial arithmetic medians

下标	原生/s	倍增/s	Lucas/s	两平方/s
10^7	0.022	0.044	0.033	0.026
10^8	0.334	0.659	0.483	0.400
5×10^8	2.015	4.013	2.927	2.359
10^9	3.844	7.844	5.622	4.610

注：每种算法、每个规模各测 5 次，表中仅列中位数。Q1、Q3、IQR 及范围见补充 arithmetic_summary.csv；四分位数采用线性插值。

2 十进制输出与进程诊断

2.1 位数与补零策略

分治输出需要先得到计划十进制位宽。GMP 和 gmpy2 对非二次幂进制的位数估算具有误差边界，gmpy2 的 num_digits 可能比真实位数多一位[8]。如果直接用这个估计进行补零，文件可能出现多余前导零。本文程序利用 MPFR 高精度浮点运算[9]，采用 256 位精度计算斐波那契位数候选值，并显式向下取整。

$$D_n = \lfloor n \log_{10}\varphi - \log_{10}\sqrt{5} \rfloor + 1$$

$$\varphi = \frac{1+\sqrt{5}}{2}$$

MPFR 提供明确的精度和舍入模式[10]。程序对第 0 项和第 1 项进行专门处理。第 100 亿项位数公式经 256 位和 1024 位 MPFR 计算，取整前均为 2,089,876,402 加小数部分；1024 位结果的小数部分为 0.1503023355…，两种精度的数值差约 1.11×10^−68。输出长度为 2,089,876,403 位，与候选值一致；这项数值对照仍不等于形式化区间证明。固定浮点精度不等于对任意大下标都完成了位数的形式化认证；更大规模应增加精度和区间边界检查。

2.2 分治转换与逐块输出

对待转换整数及计划位宽，以十的半宽次幂作除数，分成高位块和低位块。

$$v = q10^h + r,\ h = \lfloor \frac{w}{2} \rfloor,\ 0 \le r < 10^h$$

先写高位块，再按半宽位数写低位块。低位块不足计划位宽时补零，保证连接后的数字顺序与位置正确。递归到不超过 400 万位后，才调用叶块字符串转换。相同任务中的十进制幂复用，任务结束后释放缓存。GMP 的进制转换说明给出了相关分治思路[11]，本项目据此组织应用层转换与写盘。

在目标位宽下，二分树产生 1,024 个叶块。叶块策略避免了完整二十一亿字符字符串的同时驻留，但不代表整个算法只占用 400 万字符的内存。调用者仍持有完整整数，递归栈中待处理的低位块、缓存幂和除法工作区也需要内存。评价峰值应包含这些对象，并分别记录运算和转换阶段。

2.3 独立 worker 与诊断日志

界面和计算任务使用独立进程。worker 通过结构化消息报告阶段、当前层级和结果状态，并在平方开始与完成时保存操作数位数。诊断记录逐条刷新和落盘，底层库硬退出后仍能定位最后一次开始的操作。进程边界及退出码由 Python subprocess 接口管理[12]，而不是指望 Python 异常捕获能够恢复已经终止的本地进程。

项目也经历过把矩阵运算拆给多个进程的尝试。由于大整数需要序列化和传输，该方案没有成为最终后端。Python

多进程文档说明了进程间对象传送的序列化行为[13]，但本文未保留该失败路线的完整定量测量，不据此得出所有大整数并行实现都无益的结论。

诊断记录的落盘与结果文件的耐久性是两个问题。当前代码对日志使用同步刷新，但对完整数字文件没有记录单独的持久化同步时间。论文中的“转换与写盘”表示转换、应用层写入和关闭文件的阶段时间，不等同于已经测得磁盘介质持久化时间。



图 1　实测执行与核验流程。两平方重复与直接库调用对照分别计数；一致性检查不构成独立正确性证明。

Fig. 1　Measured execution and verification workflow. Trial counts distinguish two-square runs from the direct-library control; agreement checks are not an independent correctness proof.

3 实验记录与全文验证

3.1 环境与重复测量协议

2026 年 10 月 15 日，在 Intel Core i5-14400F 电脑上开展新实验。Windows 记录 10 个物理核、16 个逻辑处理器，可见物理内存 34,072,674,304 字节，系统版本为 Windows 11 build 27766。失败路径使用 Python 3.13.7，分别测试 gmpy2 2.3.1 和隔离安装的官方 2.2.1 二进制包。两者均报告 GMP 6.3.0，但并非同一 DLL。2.2.1 隔离 PyPI wheel 内的 `libgmp-10.dll` SHA-256 为 `fb2d312ca861c18e8b23648ac8440b0b613b1930ceff58f48b48ba2d3bd6b186`，不同于 2.3.1 所附 DLL；wheel 元数据不足以确定编译器和全部构建参数。WSL2 Ubuntu 使用 Python 3.12.3、gmpy2 2.1.5 和 GMP 6.3.0，内核版本 5.15.167.4，可见内存 16,240,976 KiB、交换空间 4 GiB。运行时与构建同时改变，因此不能把比较解释为单独的操作系统因果实验。

实验调用原 v4.3 源码中的算术与输出函数。源码 SHA-256 前 16 位为 f40028d222e33178（完整值见补充清单），与固定 Git 版本经 LF/CRLF 规范化后相同。四种算法在第 1000 万、1 亿、5 亿和 10 亿项各测 5 次，共 80 次。每次启动独立进程，按重复轮次循环轮换算法顺序，顺序执行全部计算。算术计时关闭应用层逐层诊断回调（diag=None），不计模块导入、进程启动和原 worker 的逐层日志开销。正文表列中位数，补充 arithmetic_summary.csv 列线性插值四分位数及范围，不据 5 次观测进行显著性推断。

Linux 采用 perf_counter 分开计量整数运算、十进制转换与关闭文件；随后额外执行 fsync 并单独计时，核验时间不计入前述阶段。驱动程序调用原分治输出函数，写出不含任务头部的纯数字文件。resource.getrusage 与/usr/bin/time -v 记录内核维护的整进程累计峰值 RSS。算术结束时的峰值和最终峰值不是相互独立的阶段峰值。Windows MPIR 使用原 v1.1.0 发布 EXE，外部计时覆盖进程创建至退出；其内部状态时间只有整秒精度，不替代 Linux 精确阶段计时。父进程每 50 ms 读取工作集、私有提交及操作系统峰值计数器；最后一次采样后短暂出现的峰值仍有漏记边界。

除 80 次算法比较外，WSL2 两平方在第 80 亿项运行 3 次仅算术实验；在第 100 亿项运行 3 次完整输出，首轮之后运行 1 次 Windows MPIR 百亿项完整输出；另补 5 次 MPIR 十亿项完整输出及 80 亿、100 亿项各 3 次纯算术重复。最后直接调用 Linux gmpy2.fib 生成第 100 亿项新对照文件，绕过应用层沿用的 Windows 分派阈值。所有环境、命令与原始指标单独保存。旧记录中的 53.6 s、527.8 s 及约 5 GiB 保留为历史材料，不再作为本次重复测量的结果。

3.2 可重复失败与执行路径对照

Windows 下 gmpy2 2.3.1 与 2.2.1 各运行 2 次。四次均在第 32 层第一次平方退出，操作数长 2,776,967,654 位，尚未进入十进制转换。stderr 均显示“GNU MP: Cannot allocate memory (size=4294597664)”。本次 2.3.1 退出码为 3，与旧报告中的 0xC0000409 分开记录；均非监测器主动终止。2.3.1 两次失败的观测峰值工作集和私有提交中位数分别为 1.82、2.46 GiB；2.2.1 两次分别为 1.81、2.46 GiB，两个计数器分别记录。2.3.1 两次重跑在故障前的最后一条日志显示可用物理内存仍超过 10 GiB；物理内存计数本身不能解释失败，需结合全宽分配记录。

另设分配器跟踪实验，在导入 gmpy2 前安装透传回调，以 size_t 全宽记录请求，并转交原分配函数。跟踪实验独立保存，不纳入性能统计。捕获的最大请求为 18,446,744,073,709,181,984 字节。该值接近 64 位 size_t 上限，其低 32 位为 4,294,597,664，与 stderr 显示值相同。实际请求等于 $2^{64}-369,632$ 字节；同一位模式按有符号 64 位整数解释为−369,632。约 4 GiB 只是低 32 位显示假象。

2026 年 10 月 16 日在受保护的独立进程中平方稀疏数 x=2^(b−1)。二分记录到相邻结果：b=2,147,352,576 成功，b=2,147,352,577 失败。首次失败输入等于 $2^{31}-2^{17}+1$ 位，平方结果按数学性质为 4,294,705,153 位，即 67,104,769 个 64 位 limb。GMP 文档中的`mpz_t`尺寸字段以 limb 计数[14]；67,104,769 远低于 2,147,483,647，故不能把这次失败解释为

触及通用 limb 上限。绕开 gmpy2 直接调用 DLL，并用 Visual Studio 2019 x64 单独编译 C 复现器，均得到同一对相邻结果。C 核心调用依次为 `mpz_init_set_ui(x,1); mpz_init(y); mpz_mul_2exp(x,x,b-1); mpz_mul(y,x,x);`，完整受保护程序随附。2.3.1 所测文件为 `C:/Users/lusun/AppData/Roaming/Python/Python313/site-packages/gmpy2.libs/libgmp-10.dll` ，SHA-256 为 `829adcf025d22e641c6816b431fbe5b226a39b390c7205192d480151646fe9c9` 。2.2.1 的另一份 DLL（SHA-256：`fb2d312ca861c18e8b23648ac8440b0b613b1930ceff58f48b48ba2d3bd6b186`）也在该点失败，完整分配请求同为 $2^{64}-523,232$ 字节。这些结果覆盖所测两份 DLL，未测得内部 FFT 取整长度，也未定位产生异常尺寸的源码表达式。故障由所测 DLL 的大整数平方/乘法路径或其分配尺寸传递触发，不依赖斐波那契递推、十进制输出或 Python 绑定。现有证据确认分配尺寸回绕；具体表达式和内部乘法分支仍待带符号的原生调用栈定位。独立复现器、完整分配记录和两份 DLL 哈希见补充材料。该相邻结果是稀疏输入的实测边界，不等于已穷举证明的全局最小失败位数；FFT 取整长度未测得。

表 2　所测 DLL 稀疏数平方的相邻实测结果

Table 2　Adjacent observed outcomes for sparse-operand squaring in the tested DLL

操作数位数 b	结果位数	64 位 limbs	实测结果
2,147,352,576	4,294,705,151	67,104,768	成功
2,147,352,577	4,294,705,153	67,104,769	分配尺寸回绕

在相同下标，WSL2 三次完成，算术时间中位数为 48.18 s，累计峰值 RSS 中位数为 2.81 GiB。Windows MPIR 发布版还完成 1 次更大的百亿项完整输出；计时专用构建另完成 80 亿、100 亿项各 3 次纯算术重复。这些数据证明两条可行的执行路径，尚未证明原 GMP 内部哪个尺寸表达式存在错误，也不推广为所有 Windows 构建的统一结论。

3.3 完整输出与新对照文件

每次 Linux 整数结果均与普通 Python 整数实现的模快速倍增比较，模数为 1,000,000,007、1,000,000,009、2,147,483,647 和 10^{100}。模核验验证若干算术关系，不能单独证明全部数字正确。新原生对照文件再次流式核验：十进制数字和给出模 9 余 6、模 11 余 0，与小整数递推一致；在 3 个 64 位以内的素数模下 Cassini 恒等式成立。周期法独立得到末 5 位 46875，90 位十进制对数计算得到首 5 位 14135。其余 4 份新输出与对照文件 SHA-256 相同。完整输出另行统计位数、检查非数字字符并分块计算 SHA-256。

两条修复路径的 4 份新输出及直接 Linux GMP 调用生成的对照文件，均含 2,089,876,403 位十进制数字。去除 MPIR 任务前缀后全文比较一致，摘要如下。

```
6fa62cdd9d65374354899719d9a58122
2c1d497492b9bcddcb859c32bc469662
```

数字前 5 位为 14135，后 5 位为 46875。新对照文件保留生成程序、命令、运行时、资源记录和输出摘要，形成可复现的生成链。它由作者本次生成，不是第三方认证标准；与被测 Linux 路径共享 GMP 及既有递推原理，因此不夸大实现独立性。

旧文件仅用于历史材料核查，不作为主要验证依据。其生成机器、原始命令及第三方来源仍未认证；完整核查记录见补充 historical_provenance.md。

表 3　新输出与对照文件核验

Table 3　New output and reference verification

核验项	结果
新完整输出	3 次 WSL2+1 次 MPIR+1 份原生对照
每份数字位数	2,089,876,403
新原生对照	Linux gmpy2.fib
全文比较	MATCH
SHA-256	相同，见正文
前 5 位 / 后 5 位	14135 / 46875
日期	2026-10-15

4 结果与讨论

第 100 亿项的 3 次 WSL2 完整实验中，算术时间中位数为 56.95 s，转换与关闭文件为 551.98 s，两阶段和中位数为 608.93 s。输出占比中位数为 90.65%。整进程累计峰值 RSS 中位数为 5.10 GiB。原始 CSV 保存每次观测、范围和额外 fsync 时间。

表 4　Linux 完整输出的新测量记录

Table 4　New complete Linux output trials

运行	运算/s	输出/s	阶段和/s	峰值/MiB
Native	48.12	542.81	590.94	5219.6
2S-1	58.60	557.22	615.81	5219.7
2S-2	53.88	548.49	602.37	5219.4
2S-3	56.95	551.98	608.93	5221.6

注：Native 是在同一 WSL2/Linux 环境直接调用 gmpy2.fib 的 1 次对照，不计入 2S-1/2/3 的中位数、四分位数和图中误差线。各列独立保留 2 位小数；阶段和先由未舍入值相加再舍入，显示分项之和可相差 0.01 s。峰值为累计 RSS，额外 fsync 及核验另计。

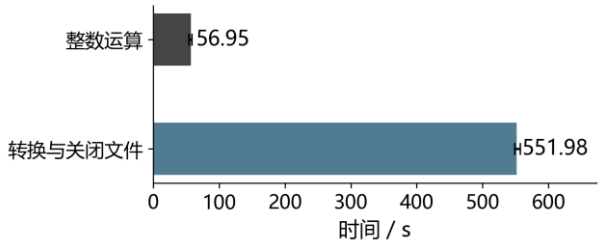


图 2　百亿项 WSL2 两平方完整输出 3 次的中位数与观测范围，不含 Native 对照；转换含关闭文件，fsync 另计。

Fig. 2　Three WSL2 two-square full-output trials at n=10^{10}; bars are medians and error bars the observed range. The Native control is excluded. Conversion includes close; fsync is separate.

Windows 原生 MPIR v1.1.0[15]一次百亿项完整输出的外部进程总时间为 2592.93 s，观测峰值工作集为 8.05 GiB。发布 EXE 的 SHA-256 前 16 位为 bda33f884b986765（完整值见补充清单）。本次直接运行该二进制，未把当前 C 文件默认为该 EXE 的重新构建证明。MPIR 总时间包含位数校正、初始化、状态线程和文件关闭；未额外增加持久化同步测量。由于端点不同，不把它与 Linux 阶段和直接换算为受控加速比。

新实验覆盖一个主机与若干具体构建。后台负载、文件缓存和固定轮换顺序仍影响外部推广；Windows 工作集、私有提交与 Linux RSS 也不混为同一指标。分配尺寸回绕已由直接 DLL 调用复现，具体源码表达式仍缺少与二进制匹配的原生调用栈。新对照文件具有本次生成链，但共享 GMP 与递推原理；旧文件的原始超算执行与再发布许可仍待证实。

保留的 C 源码与 Python 输出器均采用 4,000,000 位叶块，但 C 调用 MPIR 的 mpz_tdiv_qr、mpz_get_str 和 fwrite；Python 调用 GMP 的 f_divmod、digits 并通过 Python 文件对象写出。MPIR 发布版另执行位数校正和状态报告。Linux 写入 WSL2 的/tmp，Windows 写入 C 盘。现有记录没有把进制转换、补零、运行库 I/O 和存储等待分别计时，因此不能把总耗时差归于其中任一因素；保留的 C 源码也不是发布 EXE 的二进制匹配重建。

鉴于首轮 MPIR 转换耗时较长，实测期间调整后续组合：百亿项完整输出保留 1 次；十亿项完整输出重复 5 次；80 亿项和 100 亿项纯算术各重复 3 次。计时专用 EXE 由保留的 C 源码构建，在原 Fibonacci 函数前后加入 QueryPerformanceCounter，并在算术结束后提前退出；算术函数未改动。两个大规模的算术时间中位数分别为 107.97 s 和 122.22 s。它与发布 EXE 分别保存源码、编译命令、静态库和二进制哈希。单次 MPIR 百亿项完整输出不写成重复统计。首轮计算进程保持不变，采集器中途接管；两段内存记录与协议调整说明随原始数据保留，该次外部时间采用操作系统进程创建时间至观测退出。

表 5　MPIR 分口径实测结果与样本数

Table 5　MPIR measurements by endpoint and sample size

下标	模式	次数	时间/s	工作集/GiB
10^9	完整	5	185.50	0.78
8×10^9	算术	3	107.97	5.18
10^{10}	算术	3	122.22	6.47
10^{10}	完整	1	2592.93	8.05

5 结语

本次以重复实验记录 Windows GMP 的固定失败点，并验证 WSL2 与 Windows 原生 MPIR 两条完整输出路径。WSL2 三次与 MPIR 发布版一次生成百亿项数字，与带生成链的新 Linux GMP 对照文件全文一致。输出占 WSL2 阶段总时间的中位数为 90.65%。本实验表明，库构建与接口、临时内存和输出流程决定既有算法能否交付完整结果。工程优化和算法选择应共同接受实际数据检验。

新输出与直接库调用结果全文一致，模核验也通过了所检查的算术关系。这些结果属于一致性证据；直接对照与被测 Linux 路径共享 GMP 及既有递推原理，SHA-256 相同也不单独证明数字正确，尚未获得独立实现或形式化证明。

数据与软件可用性

源代码按 MIT 许可证公开，固定版本见文献[5]。本次补充实验驱动、原始 CSV、逐次环境、失败日志、10 月 16 日独立平方与直接 DLL 调用记录、源码与二进制哈希、数值比较清单和新对照生成命令随修订材料保存。完整大文件在本地保留并以哈希标识，尚未声称完成公开 DOI 归档。版本化软件引用见文献[16]。

作者与人工智能工具说明

陆旭东，华东师范大学附属枫泾中学。通信邮箱：lusunrise@126.com。人工智能工具参与软件开发协作、实验编排与证据处理、稿件整理和英文翻译，具体工具使用记录见补充说明。作者负责科学判断、证据核对和最终投稿文本且对文本所有内容负责；人工智能工具不作为作者。

参考文献

[1] BRENT R P, ZIMMERMANN P. Modern Computer Arithmetic[M]. Cambridge: Cambridge University Press, 2010. https://doi.org/10.1017/CBO9780511921698

[2] THE GNU MP DEVELOPERS. Fibonacci Numbers Algorithm: GNU MP 6.3.0[EB/OL]. (2023)[2026-10-14]. https://gmplib.org/manual/Fibonacci-Numbers-Algorithm

[3] THE GNU MP DEVELOPERS. FFT Multiplication: GNU MP 6.3.0[EB/OL]. (2023)[2026-10-14]. https://gmplib.org/manual/FFT-Multiplication

[4] THE GNU MP DEVELOPERS. Memory Management: GNU MP 6.3.0[EB/OL]. (2023)[2026-10-14]. https://gmplib.org/manual/Memory-Management

[5] LU X. Fibonacci 10 Billion Calculator: Python v4.3[CP/OL]. (2026)[2026-10-14]. Git revision f0c5f2488432524d6edb781cbabe1bbca2a79b6d. https://github.com/sr-lxd/fibonacci-10-billion-calculator/tree/f0c5f2488432524d6edb781cbabe1bbca2a79b6d

[6] SOFRONIOU M. mpn_mul_fft type overflow issue[EB/OL]. (2013-09-18)[2026-10-16]. GMP developers mailing list. https://gmplib.org/list-archives/gmp-devel/2013-September/003365.html

[7] THE GNU MP DEVELOPERS. Number Theoretic Functions: GNU MP 6.3.0[EB/OL]. (2023)[2026-10-14]. https://gmplib.org/manual/Number-Theoretic-Functions

[8] THE GMPY2 CONTRIBUTORS. Multiple-precision integers: gmpy2 documentation[EB/OL]. [2026-10-14]. https://gmpy2.readthedocs.io/en/latest/mpz.html

[9] FOUSSE L, HANROT G, LEFÈVRE V, et al. MPFR: A multiple-precision binary floating-point library with correct rounding[J]. ACM Transactions on Mathematical Software, 2007, 33(2): 13. https://doi.org/10.1145/1236463.1236468

[10] THE MPFR DEVELOPERS. GNU MPFR 4.2.2 manual[EB/OL]. (2025)[2026-10-14]. https://www.mpfr.org/mpfr-current/mpfr.html

[11] THE GNU MP DEVELOPERS. Binary to Radix: GNU MP 6.3.0[EB/OL]. (2023)[2026-10-14]. https://gmplib.org/manual/Binary-to-Radix

[12] PYTHON SOFTWARE FOUNDATION. subprocess: Subprocess management. Python 3.13 documentation[EB/OL]. [2026-10-14]. https://docs.python.org/3.13/library/subprocess.html

[13] PYTHON SOFTWARE FOUNDATION. multiprocessing: Process-based parallelism. Python 3.13 documentation[EB/OL]. [2026-10-14]. https://docs.python.org/3.13/library/multiprocessing.html

[14] THE GNU MP DEVELOPERS. Integer Internals: GNU MP 6.3.0[EB/OL]. (2023)[2026-10-16]. https://gmplib.org/manual/Integer-Internals

[15] THE MPIR CONTRIBUTORS. MPIR Windows source and Win64 definitions[EB/OL]. [2026-10-14]. https://github.com/BrianGladman/mpir/blob/5e0c2061af105c151970d41c8394ce956f77e455/gmp-h.in

[16] SMITH A M, KATZ D S, NIEMEYER K E, et al. Software citation principles[J]. PeerJ Computer Science, 2016, 2: e86. https://doi.org/10.7717/peerj-cs.86